

Phase 5.4

STM32 Takes Over LTE Control

Option B1: STM controls EVERYTHING — no Arduino in the toolchain

This document covers the wiring, CubeMX configuration, and pin assignments needed to put the STM32 in **direct control** of the T-A7608 modem on the LilyGO board. The ESP32 is held in reset and stays out of the way; STM32 handles modem power-on AND UART communication.

Submission 1 scope (no encryption): two devices register with the relay over LTE, exchange plain audio bytes both directions, identify each other by phone number. Encryption is added in Submission 2 (Phase 7).

#	STM32 pin	LilyGO pin	Function
2	PB1 (output)	IO12 (left header)	Modem POWERON — drive HIGH always (powers the A7608 chip)
3	PB2 (output)	IO4 (left header)	Modem PWRKEY — initially HIGH; pulse LOW for 1 second at boot to power on the modem

3.2 — UART to modem (2 wires + ground)

#	STM32 pin	LilyGO pin	Direction
4	PA9 (USART1_TX)	IO26 (left header)	STM32 → modem (RX into modem)
5	PA10 (USART1_RX)	IO27 (left header)	STM32 ← modem (TX from modem)
6	GND	GND	Common ground (mandatory)

3.3 — User buttons (2)

#	STM32 pin	Connected to	Function
7	PA1 (input, pull-up)	Call button → GND	Pressed → initiate a call to peer
8	PB0 (input, pull-up)	Answer/Hangup button → GND	Pressed → answer incoming OR end ongoing call

Buttons use STM32's internal pull-up resistors. With the button NOT pressed, the pin reads HIGH. When pressed, the button connects the pin to GND, and the pin reads LOW. Software detects the falling edge as a button press.

3.4 — Power for each board

Board	Power source	Notes
STM32 Black Pill	USB-C from PC	Same as before — for programming and runtime
LilyGO LTE board	USB-C from phone charger OR 18650 battery OR VBUS pin	Needs separate ≥5V/2A supply (USB-C from laptop is OK for testing)

Both boards must share the same GND (covered by wire 6 above). All other wires use the same ground reference automatically.

4. Complete STM32 Pin Map (after Phase 5.4)

Pin	Function	Connection	Phase added
PA0	ADC1_IN0	MAX9814 OUT (mic)	2
PA1	GPIO input (pull-up)	Call button → GND	5.4 (NEW)
PA8	GPIO output (low)	LilyGO RST (ESP32 reset hold)	5.4 (NEW)
PA9	USART1_TX	LilyGO IO26 (modem RX)	5.4 (NEW)
PA10	USART1_RX	LilyGO IO27 (modem TX)	5.4 (NEW)
PA13	SWDIO	ST-Link	1
PA14	SWCLK	ST-Link	1
PB0	GPIO input (pull-up)	Answer/Hangup button → GND	5.4 (NEW)
PB1	GPIO output (high)	LilyGO IO12 (POWERON)	5.4 (NEW)
PB2	GPIO output (high, pulse low)	LilyGO IO4 (PWRKEY)	5.4 (NEW)
PB6	I2C1_SCL	MCP4725 SCL	3
PB7	I2C1_SDA	MCP4725 SDA	3
PC13	GPIO output (LED)	Onboard LED (status)	2
3V3 / GND	Power	MAX9814, MCP4725 (3V3); LM386 (5V)	1+

5. CubeMX Configuration Changes

Open **MicDAC.ioc** in CubeMX. Do NOT touch existing peripherals (ADC1, TIM2, TIM3, I2C1) — only ADD the items below.

5.1 — Enable USART1

- Connectivity → **USART1** → Mode: **Asynchronous**.
- Parameter Settings → Baud Rate: **115200**; Word Length: 8 bits; Parity: None; Stop Bits: 1.
- Hardware Flow Control: None.
- **NVIC Settings** tab → tick **USART1 global interrupt**.
- PA9 (TX) and PA10 (RX) auto-assign on the chip diagram.

5.2 — Add 3 GPIO outputs (modem + ESP32 control)

On the chip diagram, left-click each pin and select **GPIO_Output**. Then in System Core → GPIO, click each pin row and configure:

Pin	User Label	Output level (initial)	Mode	Speed	Pull
PA8	ESP32_RST	Low	Push Pull	Low	No
PB1	MODEM_POWERON	High	Push Pull	Low	No
PB2	MODEM_PWRKEY	High	Push Pull	Low	No

Initial levels: PA8=Low keeps the ESP32 in reset from the moment STM32 boots. PB1=High keeps modem power on. PB2=High is the idle state of PWRKEY (firmware pulses it LOW for 1 second once at

boot to power on the modem chip).

5.3 — Add 2 GPIO inputs (buttons)

Left-click each pin and select **GPIO_Input**. Then in GPIO config:

Pin	User Label	Mode	Pull-up/Pull-down
PA1	BTN_CALL	Input	Pull-up
PB0	BTN_ANSWER	Input	Pull-up

5.4 — Generate code

- Project Manager → confirm Toolchain = CMake.
- Click **GENERATE CODE**.
- VS Code reloads the project automatically.

6. Verification Before Writing Firmware

After regenerating the CubeMX code, do these quick checks:

6.1 — Open Core/Src/gpio.c

Verify the `MX_GPIO_Init()` function configures all the new pins. Search for the lines that set `GPIO_PIN_8` / `GPIO_PIN_1` / `GPIO_PIN_2` etc. and confirm initial output levels and pull settings.

6.2 — Open Core/Src/usart.c

Should contain a `MX_USART1_UART_Init()` function with baud rate 115200 and PA9/PA10 configured as alternate function (USART1 TX/RX).

6.3 — Compile with no errors

- Build (Ctrl+Shift+B in VS Code).
- Should report **0 errors, 0 warnings**.
- If there are errors, the CubeMX changes had a problem — re-check.

6.4 — Multimeter sanity check (optional)

Probe	Expected reading	Why
STM32 PA8 vs GND	~0 V	ESP32 hold is LOW
STM32 PB1 vs GND	~3.3 V	POWERON is HIGH (modem powered)
STM32 PB2 vs GND (steady)	~3.3 V	PWRKEY idle HIGH
STM32 PA1 vs GND, button NOT pressed	~3.3 V	pull-up active
STM32 PA1 vs GND, button PRESSED	~0 V	button shorts to GND
STM32 PB0 same test	same as PA1	

7. What's Next — Phase 5.4 Firmware

After wiring + CubeMX is done, the firmware additions to `main.c` will include:

Block	What it does
Modem power-on sequence at boot	Pulse PB2 (PWRKEY) low for 1 sec, wait 8 sec for boot
UART RX interrupt handler	Buffers bytes received from the modem so the main loop can parse responses
<code>send_at(cmd, expected, timeout)</code>	Helper that writes an AT command, waits for the expected response within a timeout
LTE bring-up state machine	Runs AT, ATE0, AT+CPIN?, AT+CSQ, AT+CGDCONT, AT+CGACT, AT+NETOPEN, AT+CIOPEN to relay
REG sender	Sends "REG " through the open TCP socket, waits for "OK"
Status LED logic	PC13: slow-blink = registered idle, fast-blink = ringing, solid = in call
Button polling	Detects PA1 (Call) and PB0 (Answer/Hangup) presses with debouncing

All of this gets ADDED to the existing MicDAC project. The Phase 3 audio code (ADC sampling, DAC playback) keeps running unchanged in the background.

8. Action Items Before Continuing

- Buy 4 push-buttons (4-leg tactile) if not already purchased — 2 per device.
- Wire the 8 connections per Section 3 on Device 1's breadboard.
- Power the LilyGO board separately from the STM32 (USB-C charger or 18650 battery for LilyGO; STM32 keeps its USB-C from PC).
- In CubeMX: open MicDAC.ioc, add USART1 + 3 GPIO outputs + 2 GPIO inputs as in Section 5.
- Generate code, build with 0 errors.
- Multimeter check the static GPIO levels (Section 6.4).
- Once everything verified — ready for the firmware code (Phase 5.4 part 2).

End of Phase 5.4 hardware/CubeMX setup document. Firmware code will follow in Phase 5.4 Part 2.